Proyecto Final Etapa 3: Análisis de datos y backend

Autores: Génova, Candela – Massa, Luca – Kristensen, Franco

Docente: Fernandez Carbonell, Cesar Augusto

Facultad: Universidad Tecnológica Nacional (UTN) Extensión Áulica Necochea.

Materia: Programación IV

07/10/2026



Persistencia de datos

A partir del análisis realizado en la **Etapa 1** y de las pantallas diseñadas en la **Etapa 2**, se determinó que información necesita persistir y que tecnología resulta adecuada para almacenarla.

En la **Etapa 1** se identifica como necesidad principal centralizar la información institucional, los horarios, talleres, actividades, sedes y canales de contacto. También se plantea un espacio restringido para el personal, destinado a la gestión de participantes, asistencia, becas, documentación y seguimineto.

La **Etapa 2** muestra estas necesidades mediante pantallas públicas y privadas. El usuario visitante puede consultar talleres, noticias, contacto, sugerencias y utilizar el buscador, mientras que los usuarios registrados pueden gestionar su perfil e inscribirse a talleres. Por otra parte, el administrador puede gestionar alumnos, barrios, asistencias, equipo, seguimiento, actividades y solicitudes de inscripción.

Por lo tanto, esta persistencia se considera por las características de los datos, sus relaciones, las operaciones necesarias y los requerimientos del sistema.

3.1 Entidades del sistema.

Las entidades son determinadas a partir de la información que aparecen en las pantallas de la Etapa 2

Pantalla / funcionalidad:	Información que necesita persistir:
Registro / Inicio de Sesión	Usuario, correo, contraseña, rol
Gestión de perfil	Datos del usuario
Talleres	Taller, inscripción, horario, barrio, actividad
Formulario de inscripción	Usuario, talleres, datos personales, estudios, colegio, DNI, comentario y estado
Mis inscripciones	Inscripción y estado
Solicitudes de inscripción	Inscripcion, usuario, taller y estado
Gestión de alumnos	Datos de los alumnos
Asistencia	Alumno, taller/actividad, fecha
Seguimiento de alumnos	Alumnos, profesor, observación, fecha
Gestión del equipo	Personal, función/rol y datos de contacto

Gestión de barrios	Barrios, talleres/actividades disponibles
Actividades diarias	Actividad, fecha, taller/ barrio y estado
Noticias	Titulo, contenido, fecha
Contacto y ubicación	Sedes, dirección, ubicación y redes sociales
Sugerencias	Usuario/contacto, mensaje y fecha
Buscador R4	Información indexable de talleres, noticias, actividades, barrios, etc.

Entidades propuestas:
1- **Usuario**: Representa a las personas que poseen una cuenta en el sistema.
Atributos:

- id_usuario — PK.
- username
- email
- contraseña
- rol
- fecha_registro
- estado(activo/inactivo)

El atributo rol va a permitir diferenciar al usuario común del administrador. En la Etapa 2 se establece que el registro genera inicialmente un usuario común y que el acceso administrativo es restringido.

2-**Alumno**: Representa la información específica de los participantes que realizan una inscripción.
Atributos:

- id_alumno — PK
- id_usuario — FK
- nombre
- apellido
- email
- telefono
- dni
- tiene_beca
- estudios_completos
- colegio

La separación entre Usuario y Alumno permite que no todos los usuarios del sistema sean necesariamente alumnos.
La gestión administrativa contempla visualizar el perfil de los alumnos y su información, mientras que la solicitud permite consultar los datos enviados y las imágenes del DNI.

3-**Barrio**: Representa los barrios donde se desarrollan las actividades.
Atributos:

- id_barrio — PK
- nombre
- descripcion

4-**Taller**: Representa cada propuesta disponible para los usuarios.
Atributos:

- id_taller — PK
- nombre
- descripcion
- dia
- hora_inicio
- hora_fin
- edad_minima
- edad_maxima
- id_barrio — FK
- cupo(disponible/agotado)
- estado_taller(activo/inactivo)

Esta entidad surge directamente de la pantalla de Talleres y de la necesidad de mostrar horarios y talleres segmentados por edad.

5-**Inscripción**: Representa la solicitud de un alumno para participar de un taller.
Atributos:

- id_inscripcion — PK
- id_alumno — FK
- id_taller — FK
- fecha_solicitud
- estado_inscripcion
- comentario
- fecha_respuesta

El atributo estado puede tomar valores como:

- **Pendiente**
- **Aceptada**
- **Rechazada**

La pantalla de solicitud del administrador permite aceptar o rechazar las inscripciones.

6-**Asistencia**: Registra la asistencia de un alumno.
Atributos:

- id_asistencia — PK
- id_alumno — FK
- id_taller — FK
- fecha
- presente

7-**Personal**: Representa profesores, coordinadores, tutores u otros integrantes del equipo.
Atributos:
- id_personal — PK
- id_usuario — FK
- funcion

La pantalla de gestión de equipo permite consultar los datos de profesores y administradores.

8-**Seguimiento**: Permite registrar observaciones sobre los alumnos.
Atributos:
- id_seguimiento — PK
- id_alumno — FK
- id_personal — FK
- fecha
- observacion

9-**Actividad**: Representa las actividades realizadas dentro del programa.
Atributos:
- id_actividad — PK
- nombre
- descripcion
- fecha
- id_taller — FK
- id_barrio — FK

La gestión de actividades diarias permite consultar las actividades realizadas y filtrarlas.

10-**Noticia**: Representa las novedades publicadas en el sitio.
Atributos:
- id_noticia — PK
- titulo
- contenido
- fecha_publicacion
- imagen
- estado(activa/inactiva)

11-**Sede**: Representa las ubicaciones físicas del programa.
Atributos:
- id_sede — PK

- nombre
- direccion
- descripcion

12-**Red Social**: Permite almacenar los canales oficiales.
Atributos:

- id_red — PK
- id_sede — FK
- tipo(ej: Facebook, Instagram, WhatsApp)
- URL
- estado(activo/inactivo)

13-**Sugerencia**: Representa las consultas o sugerencias desde el sitio.
Atributos:

- id_sugerencia — PK
- id_usuario — FK
- mensaje
- fecha
- estado

La pantalla de Sugerencia permite que el visitante informe errores. problemas o aspectos que podrían mejorarse.

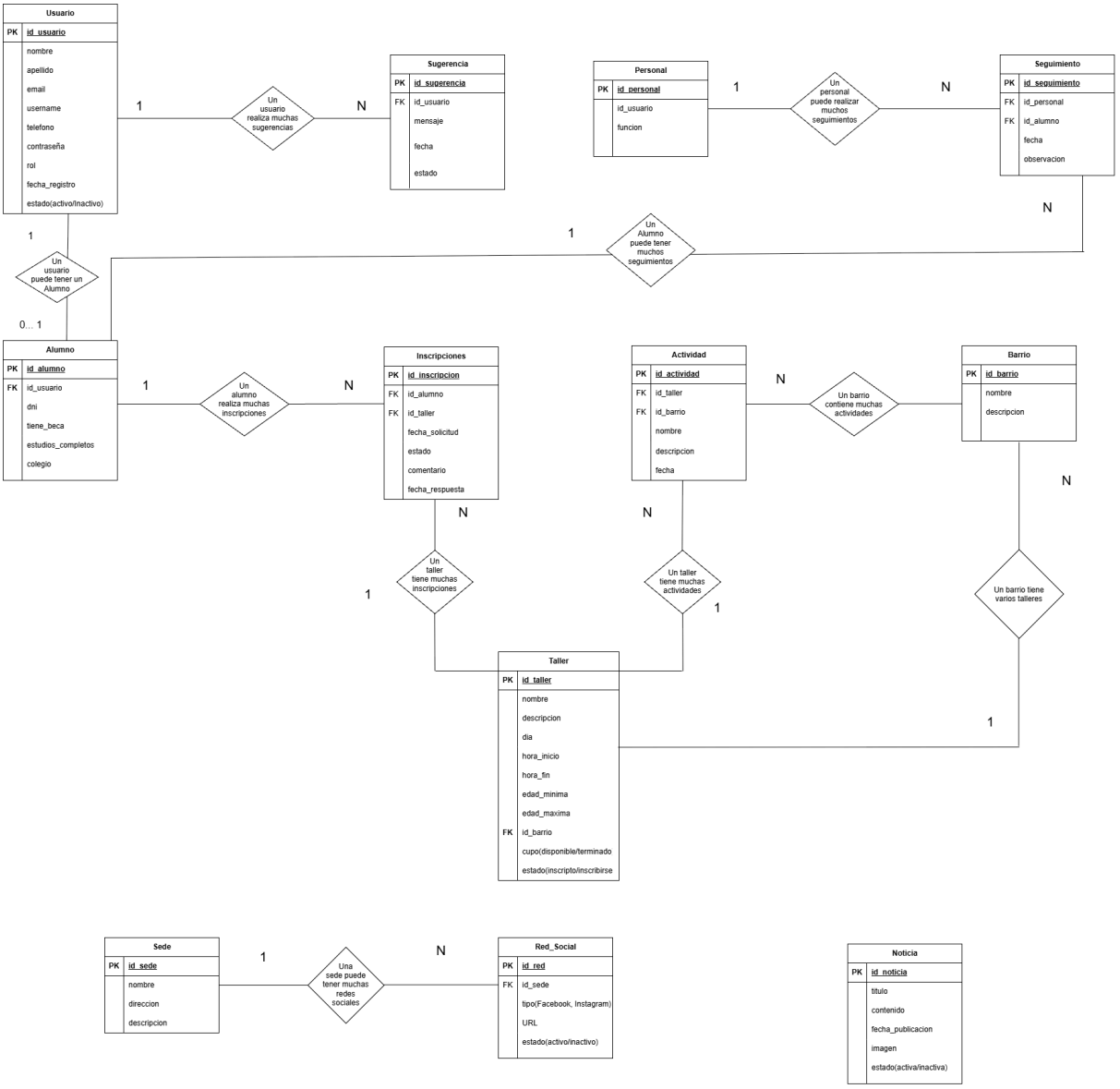
Relaciones principales:

- Un Usuario puede tener un Alumno
- Un Alumno puede realizar muchas Inscripciones
- Un Usuario puede realizar muchas sugerencias.
- Un Taller puede tener muchas inscripciones.
- Un Barrio puede tener muchos Talleres
- Un Alumno puede tener muchas Asistencias
- Un Taller puede tener muchas Asistencias
- Un Alumno puede tener mucho Seguimientos
- Un Personal puede realizar muchos Seguimientos
- Un Taller puede tener muchas Actividades
- Un Barrio puede tener muchas Actividades
- Una Sede puede tener muchas Redes Sociales

3.2 Modelo de Datos.

Seleccionamos un modelo relacional y se representa el modelo de datos mediante un diagrama **ENTIDAD-RELACIÓN**. El modelo muestra las tablas principales del sistema, sus atributos, claves primarias y foráneas y las relaciones existentes entre ellas.

Figura 1 - Diagrama entidad-relación(realizado en draw.io).



Link directo de la herramienta utilizada:
https://app.diagrams.net/#G1_G4goXxnYI_CPXGSFXXVPQZimC6eEAzy#%7B"pageId"%3A"zxPc_GmHY7BkyfeHDZW3"%7D

3.3 - Análisis comparativo de alternativas de persistencia.

Para elegir dónde almacenar la información del sistema se compararon 3 alternativas, evaluadas con criterios ponderados que surgen de las necesidades identificadas en las etapas 1 y 2 y de las entidades definidas en el punto 3.1.

ALTERNATIVAS CONSIDERADAS:

- Base de datos relacional (MySQL o PostgresSQL)
  Los datos se organizan en tablas relacionadas entre sí mediante claves primarias y foráneas, y se consultan con SQL.

- Base de datos no relacional (mongoDB), los datos se guardan en documentos similares a objetos JSON, agrupados en colecciones.

- Archivos JSON locales
  Los datos se guardan en archivos de texto dentro del servidor, que son leídos y escritos directamente desde node.

CRITERIOS DE EVALUACIÓN Y PESO

Escala de peso/prioridad: 3= alta, 2= media, 1= baja.

Escala de puntaje: 5= muy bueno, 1= muy malo.

Puntaje ponderado: puntaje X peso/prioridad del criterio.

- Relaciones entre datos e integridad (**alta**), el sistema tiene relaciones de uno a muchos (por ejemplo, un alumno realiza muchas inscripciones) y de muchos a muchos entre alumnos y talleres, resueltas a través de la inscripción. así que, se necesita que la base impida datos inconsistentes, como inscripciones a talleres inexistentes.

- Seguridad de datos sensibles (**alta**), el sistema almacena datos de menores (DNI, beca, colegio, observaciones de seguimiento) y contraseñas de usuarios, se necesita controlar los accesos y proteger la información.

- Búsqueda de contenidos (**media**), el buscador debe devolver resultados diferentes según la palabra ingresada y permitir filtros, por ejemplo por barrio o actividad.

- Consultas, filtros y reportes (**media**), el panel administrativo requiere planillas de alumnos inscriptos, filtros, asistencia por fecha y exportación a pdf.

- Integración con el stack del proyecto (**media**), que tan sencillo es conectar la alternativa con el backend desarrollado con node.

- Costo y alojamiento (**baja**), se trata de una institución municipal sin fines de lucro y con recursos limitados, por lo que interesa una solución gratuita o de bajo costo y fácil de alojar.

- Familiaridad del grupo (**baja**), experiencia previa del equipo con la tecnología, que reduce el tiempo de desarrollo y los errores.

- Mantenimiento y crecimiento (**baja**), qué tan fácil resulta que una persona de la institución o del municipio pueda mantener y ampliar el sistema.

TABLA COMPARATIVA

Cada celda indica: puntaje(puntaje ponderado)

CRITERIO	PESO/PRIORIDAD	RELACIONAL	NO RELACIONAL	JSON locales
1. RELACIONES E INTEGRIDAD	3	5(15)	2(6)	1(3)
2. SEGURIDAD DE DATOS	3	4(12)	3(9)	1(3)
3. BÚSQUEDA	2	4(8)	4(8)	2(4)
4. CONSULTAS Y REPORTES	2	5(10)	3(6)	1(2)
5. INTEGRACIÓN CON NODE	2	4(8)	5(10)	4(8)
6. COSTO Y ALOJAMIENTO	1	4(4)	4(4)	5(5)
7. FAMILIARIDAD DEL GRUPO	1	5(5)	4(4)	2(2)

8. MANTENIMIENTO Y CRECIMIENTO	1	4(4)	3(3)	1(1)
TOTAL (MÁXIMO 80)		66	50	28

JUSTIFICACIÓN DE LOS PUNTAJES

● BASE DE DATOS RELACIONAL

RELACIONES E INTEGRIDAD(15pts): Las claves foráneas y las tablas intermedias resuelven la relación de muchos a muchos de las inscripciones y evitan, por ejemplo, inscribir a un alumno a un taller inexistente.

SEGURIDAD(12pts): Permite crear usuarios de base de datos con diferentes permisos y almacenar las contraseñas aseadas. no es automático, requiere configuración.

CONSULTAS Y REPORTES(10pts): SQL resuelve filtros, uniones y conteos de forma directa. Por ejemplo, el cupo disponible de un taller se calcula como el cupo máximo menos las inscripciones aceptadas.

COSTO(8pts): MySQL y PostgreSQL son software libres. Alojarlos en la nube puede tener costo o límites en los planes gratuitos.

● NO RELACIONAL

RELACIONES E INTEGRIDAD(6pts): no impone relaciones, se pueden simular con referencias, pero la integridad queda a cargo del código, y el modelo de este sistema es relacional.

INTEGRACIÓN CON NODE(10pts): los documentos son muy parecidos a objetos de JavaScript, lo que simplifica el trabajo con node.
costo: existe un plan gratuito en la nube, aunque con límites.

● ARCHIVOS JSON LOCALES

COSTO Y ALOJAMIENTO(10PTS): Es la opción más simple y gratuita, pero no cubre relaciones, no controla accesos concurrentes, no cuenta con usuarios ni

permisos, además que cada consulta implica leer archivos completos. Dado que se almacenan datos de menores y que varios administradores pueden trabajar a la vez, no resulta adecuado para este sistema.

RESULTADO DEL ANÁLISIS

La base de datos relacional fue la que obtuvo el mayor puntaje (70 de 80), impulsada principalmente por los dos criterios de prioridad alta: relación e integridad y seguridad de datos. la base de datos no relacional (54 de 80) es otra alternativa viable y más cómoda de integrar con node, pero queda atrás con integridad de datos y reportes. Los archivos JSON locales (33 de 80) sirven para prototipos, pero no para un sistema con datos sensibles y múltiples usuarios. La decisión final y su fundamentación se desarrollan en el punto 3.4.

3.4 Decisión y fundamentación.

DECISIÓN

A partir del análisis comparativo del punto 3.3, se decide utilizar una base de datos relacional, específicamente **POSTGRESQL** como alternativa de persistencia del sistema. Esta opción obtuvo el mayor puntaje (70 de 80) frente a los 54 puntos de MongoDB y los 33 de los archivos JSON locales.

FUNDAMENTACIÓN

Dentro de las bases relacionales se eligió PostgreSQL frente a MySQL porque ofrece búsqueda de texto completo nativa con soporte para español, que el buscador de la etapa 2 necesita para devolver resultados distintos según la palabra ingresada. Además permite definir roles con permisos diferenciados, trae la extensión pgcrypto para trabajar con hashes y soporta restricciones de integridad (claves foráneas y CHECK) de forma estricta. Ambos son software libre y existen servicios de alojamiento gratuito o de bajo costo para los dos, por lo que el costo no es un factor diferenciador.

La decisión se apoya principalmente en dos criterios de alto peso: las relaciones entre datos y la seguridad de los datos sensibles.

En primer lugar, el modelo de datos definido en el punto 2 es marcadamente relacional. Un usuario tiene un alumno, un alumno puede realizar muchas inscripciones, un taller recibe muchas inscripciones y muchas asistencias, un barrio contiene muchos talleres y actividades. Una base relacional permite expresar estas relaciones mediante claves foráneas, de modo que el propio motor impida registros inconsistentes, como una inscripción a un taller que no existe o la eliminación de un alumno que todavía tiene asistencias registradas. en una base no relacional o en archivos JSON, esa validación quedaría a cargo del código de la aplicación, lo que aumenta el riesgo de errores.

En segundo lugar, el sistema almacena datos personales de menores de edad, como DNI, imágenes de documentación, situación de beca, colegio y observaciones de seguimiento, además de las contraseñas de los usuarios. Una base relacional permite crear usuarios de base de datos con permisos limitados, de modo que la aplicación no se conecte con una cuenta de administrador total. Las contraseñas se almacenan con hash, por ejemplo mediante bcrypt desde node, y nunca en texto plano. Las imágenes del dni se guardarán como archivos en el servidor, con acceso restringido, y en la base solo se registrará su ubicación. Esto resulta coherente con el deber de proteger los datos personales que establece la ley 25.326 de protección de los datos personales.

En tercer lugar, la base relacional resuelve de forma directa las consultas que requiere el panel administrativo: planilla de alumnos inscriptos, asistencia por fecha, filtros por barrios y taller y conteos. Por ejemplo, el cupo disponible de un taller se calcula como el cupo máximo menos la cantidad de inscripciones aceptadas, sin necesidad de guardar ese dato por separado.

Por último, se trata de software libre y gratuito, ampliamente documentado y con una gran comunidad, lo que facilita que otras personas del municipio o de la institución puedan mantener y ampliar el sistema en el futuro.

¿POR QUÉ SE DESCARTARON LAS OTRAS ALTERNATIVAS?

MongoDB resulta viable y más cómoda de integrar con node, pero solo supera a la base relacional por ese criterio, por una diferencia de dos puntos. En los demás criterios iguala o queda por debajo, y pierde en integridad de datos, que es lo más importante para este sistema.

Los archivos JSON locales solo superan a la base relacionada en costo y alojamiento, no controlan accesos simultáneos, no tienen usuarios ni permisos, y cada consulta implica leer archivos completos, por lo que no resultan adecuados para la protección de datos ni para varios administradores trabajando a la vez. Pueden servir para prototipos, pero no para este sistema.

LIMITACIONES ASUMIDAS

La elección de la base relacional implica aceptar algunas desventajas: la seguridad no es automática y requiere configurar usuarios, permisos y copias de respaldo. además, alojar la base en la nube puede tener costo o límites en los planes gratuitos, por lo que se evaluarán opciones de bajo costo o un alojamiento previsto por el municipio. Por último, los cambios en las estructuras de las tablas deben planificarse, aunque esto es una ventaja en un sistema donde la consistencia de los datos es prioritaria.

MOTOR ELEGIDO

Dentro de las bases de datos relacionales, se utilizara postgresSQL, un sistema de gestión de base de datos de código abierto y gratuito.

Cumple con los requisitos analizados en esta sección, soporta claves foráneas y restricciones de integridad, permite crear roles de usuario con permisos diferenciados y resuelve consultas con filtros, uniones y conteos mediante SQL. Además, es compatible con node a través de librerías como pg, y existen servicios de alojamiento gratuito o de bajo costo que lo ofrecen.

3.5 Juego de datos de prueba:

Para realizar las pruebas del sistema se propone un juego de datos compuesto por información representativa de las distintas entidades de acuerdo al punto **3.2 'Modelo de Datos'** que forman parte del Programa Envión-Quequén y Barrio Cultura. Se incluirán **usuarios**, **alumnos**, **personal**, **talleres**, **barrios**, **inscripciones**, **asistencias**, **seguimientos, actividades, noticias, sedes, redes sociales y sugerencias,** de manera que sea posible validar el funcionamiento general del sistema y las relaciones definidas en la base de datos.

El juego de datos contará con registros de prueba suficientes, distribuidos entre las diferentes tablas del sistema, permitiendo simular situaciones reales que puedan presentarse durante la utilización de la aplicación.

Por ejemplo, se incluirán contenidos relacionados con:

- **Usuarios** registrados con distintos roles(*Usuario, Personal y Administrador*).
- **Alumnos** pertenecientes a diferentes instituciones educativas.

- **Inscripciones** en estado **Pendiente, Aceptada y Rechazada.**
- **Talleres** de música, arte, deportes, actividades recreativas, entre otras.
- **Registros** de asistencia a talleres.
- **Barrios** donde se desarrollan los talleres.
- **Noticias** y actividades publicadas por el programa.
- **Sugerencias** enviadas por los usuarios.
- **Sedes y redes sociales** oficiales.
- Jornadas de salud.
- Reuniones con familias.
- Eventos del programa.

Esto permitirá realizar diferentes pruebas sobre las funcionalidades del sistema.

Caso de prueba	Resultado esperado:
Consultar talleres por barrio	Devuelve los talleres asociados al barrio seleccionado.
Consultar inscripciones pendientes	Devuelve únicamente las solicitudes pendientes de revisión.
Buscar un alumno por DNI	Devuelve la información del alumno correspondiente.
Consultar asistencias de un alumno	Devuelve los registros de asistencias asociados al alumno.
Consultar seguimientos realizados	Muestra las observaciones registradas por el personal.
Consultar noticias publicadas	Devuelve las noticias activas del sistema.
Buscar un taller inexistente	No devuelve resultados.

De esta manera, el volumen y la variabilidad de los datos permiten comprobar que el correcto funcionamiento de las consultas, las restricciones de integridad y las relaciones entre las entidades, validando el comportamiento esperado de la aplicación en distintos escenarios de uso.

Datos considerados

Para las pruebas se cargaron registros de ejemplo correspondientes a las principales entidades del sistema:

Entidad	Cantidad
Usuario	5

Alumno	2
Personal	1
Barrio	3
Taller	3
Inscripción	3
Asistencia	3
Seguimiento	2
Actividad	2
Noticia	2
Sede	2
Red Social	3
Sugerencia	2

3.5.1 - Consultas de prueba

-- pruebas.sql - Consultas de prueba (ejecutar después de schema.sql y data.sql)

1) Talleres por barrio
--Esperado: 6 filas -> Programación Inicial y Teatro (Centro), Arte (Estación Quequén), Deportes (Los Álamos), Huerta comunitaria y Música (9 de Julio)

```
SELECT t.nombre AS taller, b.nombre AS barrio
FROM taller t
JOIN barrio b ON t.id_barrio = b.id_barrio
ORDER BY b.nombre, t.nombre;
```

2) Inscripciones pendientes
Esperado: 2 filas (id_inscripcion 2 y 4)

```
SELECT * FROM inscripcion WHERE estado_inscripcion = 'PENDIENTE';
```

3) Buscar un alumno por DNI
Esperado: Ana López, Escuela N°7

```
SELECT a.id_alumno, u.nombre, u.apellido, a.dni, a.colegio
FROM alumno a
JOIN usuario u ON u.id_usuario = a.id_usuario
WHERE a.dni = '40111222';
```

4) Asistencias por alumno
Esperado: 5 filas (Ana López: 3, Sofía Díaz: 2)

```
SELECT u.nombre, u.apellido, asi.fecha, asi.presente
FROM alumno a
JOIN usuario u    ON u.id_usuario = a.id_usuario
JOIN asistencia asi ON asi.id_alumno = a.id_alumno
ORDER BY u.apellido, asi.fecha;
```

ANEXO A - Script de creación de la base de datos y tablas (schema.sql)

```
BEGIN;

CREATE TABLE usuario (
    id_usuario INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    contraseña VARCHAR(255) NOT NULL,
    email VARCHAR(30) NOT NULL UNIQUE,
    rol VARCHAR(20) NOT NULL
        CHECK (rol IN ('USUARIO','PERSONAL','ADMIN')),
    fecha_registro TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    estado BOOLEAN NOT NULL DEFAULT TRUE
);
```

```
CREATE TABLE alumno (
    id_alumno INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_usuario INTEGER,
    nombre VARCHAR(100) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    telefono VARCHAR(30),
    dni VARCHAR(20) NOT NULL UNIQUE,
    tiene_beca BOOLEAN NOT NULL DEFAULT FALSE,
    estudios_completos BOOLEAN NOT NULL DEFAULT FALSE,
    colegio VARCHAR(200),
    CONSTRAINT fk_alumno_usuario
        FOREIGN KEY (id_usuario)
        REFERENCES usuario(id_usuario)
);
```

```
CREATE TABLE personal (
    id_personal INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_usuario INTEGER NOT NULL UNIQUE,
    funcion VARCHAR(100) NOT NULL,
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT fk_personal_usuario
        FOREIGN KEY (id_usuario)
        REFERENCES usuario(id_usuario)
);
```

```
CREATE TABLE barrio (
    id_barrio INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE,
    descripcion TEXT
);
CREATE TABLE taller (
    id_taller INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL,
    descripcion TEXT,
    dia VARCHAR(30) NOT NULL,
    hora_inicio TIME NOT NULL,
    hora_fin TIME NOT NULL,
    edad_minima INTEGER NOT NULL,
    edad_maxima INTEGER NOT NULL,
    id_barrio INTEGER NOT NULL,
    cupo INTEGER NOT NULL CHECK (cupo > 0),
    estado_taller BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT fk_taller_barrio
        FOREIGN KEY (id_barrio)
        REFERENCES barrio(id_barrio)
);
```

```
CREATE TABLE inscripcion (
    id_inscripcion INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    id_alumno INTEGER NOT NULL,
    id_taller INTEGER NOT NULL,
    fecha_solicitud TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    estado_inscripcion VARCHAR(20) NOT NULL
        CHECK (estado_inscripcion IN
('PENDIENTE','ACEPTADA','RECHAZADA')),
    comentario TEXT,
    fecha_respuesta TIMESTAMP,
```

```
    CONSTRAINT fk_inscripcion_alumno
        FOREIGN KEY (id_alumno)
        REFERENCES alumno(id_alumno),
    CONSTRAINT fk_inscripcion_taller
        FOREIGN KEY (id_taller)
        REFERENCES taller(id_taller)
);
```

```
CREATE TABLE asistencia (
    id_asistencia INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_alumno INTEGER NOT NULL,
    id_taller INTEGER NOT NULL,
    fecha DATE NOT NULL,
    presente BOOLEAN NOT NULL,
    CONSTRAINT fk_asistencia_alumno
        FOREIGN KEY (id_alumno)
        REFERENCES alumno(id_alumno),
    CONSTRAINT fk_asistencia_taller
        FOREIGN KEY (id_taller)
        REFERENCES taller(id_taller)
);
```

```
CREATE TABLE seguimiento (
    id_seguimiento INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    id_alumno INTEGER NOT NULL,
    id_personal INTEGER NOT NULL,
    fecha DATE NOT NULL,
    observacion TEXT NOT NULL,
    CONSTRAINT fk_seguimiento_alumno
        FOREIGN KEY (id_alumno)
        REFERENCES alumno(id_alumno),
    CONSTRAINT fk_seguimiento_personal
        FOREIGN KEY (id_personal)
        REFERENCES personal(id_personal)
);
```

```sql
CREATE TABLE actividad (
    id_actividad INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL,
    descripcion TEXT,
    fecha DATE NOT NULL,
    id_taller INTEGER NOT NULL,
    id_barrio INTEGER NOT NULL,
    estado VARCHAR(50),
    CONSTRAINT fk_actividad_taller
        FOREIGN KEY (id_taller)
        REFERENCES taller(id_taller),
    CONSTRAINT fk_actividad_barrio
        FOREIGN KEY (id_barrio)
        REFERENCES barrio(id_barrio)
);
```

```sql
CREATE TABLE noticia (
    id_noticia INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    titulo VARCHAR(250) NOT NULL,
    contenido TEXT NOT NULL,
    fecha_publicacion TIMESTAMP NOT NULL,
    imagen VARCHAR(500),
    activa BOOLEAN NOT NULL DEFAULT TRUE
);
```

```sql
CREATE TABLE sede (
    id_sede INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL,
    direccion VARCHAR(250) NOT NULL,
    descripcion TEXT
);
```

```sql
CREATE TABLE red_social (
    id_red INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_sede INTEGER NOT NULL,
    tipo VARCHAR(50) NOT NULL,
    url VARCHAR(500) NOT NULL,
    activa BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT fk_red_sede
        FOREIGN KEY (id_sede)
        REFERENCES sede(id_sede)
);
```

```
CREATE TABLE sugerencia (
    id_sugerencia INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_usuario INTEGER,
    mensaje TEXT NOT NULL,
    fecha TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(30) NOT NULL,
    CONSTRAINT fk_sugerencia_usuario
        FOREIGN KEY (id_usuario)
        REFERENCES usuario(id_usuario)
);

COMMIT;
```

ANEXO B - Script de carga de datos de prueba (data.sql)

```
INSERT INTO usuario
(id_usuario, username, email ,contrasena, rol, fecha_registro, estado)
VALUES
(1, 'admin', 'admin@programa.org', crypt('Prueba123', gen_salt('bf',
10)), 'ADMIN',    '2026-01-01', true), (2, 'ana_lopez',
'ana@gmail.com', crypt('Prueba123', gen_salt('bf', 10)), 'USUARIO',
'2026-01-05', true), 3, 'carlos_ruiz',   'carlos@gmail.com',
crypt('Prueba123', gen_salt('bf', 10)), 'USUARIO',  '2026-01-07',
true), (4, 'maria_tutor','maria@programa.org', crypt('Prueba123',
gen_salt('bf', 10)), 'PERSONAL', '2025-12-15', true),(5, 'lucas_gomez',
'lucas@gmail.com', crypt('Prueba123', gen_salt('bf', 10)), 'USUARIO',
'2026-01-10', true), (6, 'sofia_diaz', 'sofia@gmail.com',
crypt('Prueba123', gen_salt('bf', 10)), 'USUARIO',  '2026-01-12',
true),(7, 'tomas_vega', 'tomas@gmail.com', crypt('Prueba123',
gen_salt('bf', 10)), 'USUARIO','2026-01-14', true);
```

```
INSERT INTO personal
(id_personal,id_usuario,funcion,activo)
VALUES
(1,4,'Tutor',true);
```

```
INSERT INTO barrio
(id_barrio,nombre,descripcion)
VALUES
(1,'Centro','Zona centrica'),
(2,'Norte','Zona norte'),
(3,'Sur','Zona sur');
```

```
INSERT INTO taller
(
id_taller,
nombre,
descripcion,
dia,
hora_inicio,
hora_fin,
edad_minima,
edad_maxima,
id_barrio,
cupo,
activo) VALUES (1,'Arte','Un espacio libre y gratuito para mancharse las manos, jugar con arcilla y pintar con témperas','Lunes','16:00','18:00',12,18,1,20,true),(2,'Musica','Explorá la percusión, la guitarra, el canto o la creación musical colectiva', 'Miercoles','17:00','19:00',12,18,2,15,true),(3,'Deportes','Un espacio pensado para correr, saltar, jugar en equipo y divertirse al aire libre.','Viernes','15:00','17:00',14,18,3,25,true);
```

```
INSERT INTO alumno
(id_alumno,
id_usuario,
nombre,
apellido,
telefono,
dni,
tiene_beca,
estudios_completos,
colegio)
VALUES(1,2,'Juan', 'Perez','2262354323','40111222',true,true,'Escuela N°7'),(2,3,'42555444',false, false,'Escuela Tecnica N°3');
```

```
INSERT INTO inscripcion
(
id_inscripcion,
id_alumno,
id_taller,
fecha_solicitud,
estado,
comentario,
fecha_respuesta
) VALUES (1,1,2,'2026-02-01','ACEPTADA','Documentacion correcta',
'2026-02-03'),(2,2,1,'2026-02-01','PENDIENTE',NULL,NULL),(3,2,3,'2026-0
2-02','RECHAZADA','Cupo completo','2026-02-05');
```

```
INSERT INTO asistencia
(
id_asistencia,
id_alumno,
id_taller,
fecha,
presente
)VALUES(1,1,2,'2026-03-01',true),(2,1,2,'2026-03-08',true),(3,1,2,'2026
-03-15',false);
```

```
INSERT INTO actividad
(
id_actividad,
nombre,
descripcion,
fecha,
id_taller,
id_barrio,
estado
)
VALUES(1,'Dibujo y Pintura','Actividad
introductoria','2026-03-05',1,1,'FINALIZADA'),(2,'Canto',
'Interpretación de canciones','2026-03-07',2,2,'FINALIZADA');
```

```
INSERT INTO noticia
(
id_noticia,
titulo,
contenido,
fecha_publicacion,
imagen,
activa
)VALUES(1, 'Comienzan los talleres 2026',   'Se abren las inscripciones
a los talleres de arte, música y deportes.', '2026-02-01',
'noticia1.jpg', true),(2, 'Nuevas vacantes',               'Hay cupos
disponibles en los talleres de teatro y huerta comunitaria.',
'2026-02-20', 'noticia2.jpg', true),
(3, 'Jornada de salud en el Centro', 'Se realizará una jornada de salud
abierta a las familias del programa.', '2026-03-25', 'noticia3.jpg',
true), (4, 'Reunión con familias', 'Convocamos a las familias a una
reunión para conocer las propuestas del año.',
'2026-03-28','noticia4.jpg', true),(5, 'Muestra de fin de año 2025',
'Gracias a todos los que participaron de la muestra de arte y música.',
'2025-12-10', 'noticia5.jpg', false);
```

```
INSERT INTO sede
(
id_sede,
nombre,
direccion,
descripcion
)
VALUES(1,'Sede Centro','Calle 60 N°1200','Sede principal'),(2,'Sede
Norte','Av. 59 N°500','Anexo norte');
```

```
INSERT INTO red_social
(
id_red,
id_sede,
tipo,
url,
activa
)
```

```
VALUES(1,1,'Instagram','https://instagram.com/programa',true),(2,1,
'Facebook','https://facebook.com/programa',true),(3,2,'WhatsApp',
'https://wa.me/5492262123456',true);
```

```
INSERT INTO sugerencia
(
id_sugerencia,
id_usuario,
mensaje,
fecha,
estado
)
VALUES(1,2,'Seria util agregar mas talleres de
tecnologia','2026-03-20','ABIERTA'),(2,3,'No encontraba el formulario
de inscripcion','2026-03-21','RESUELTA');
COMMIT;
```